

TOPIC 1

Experiment 1 – First Palindromic String

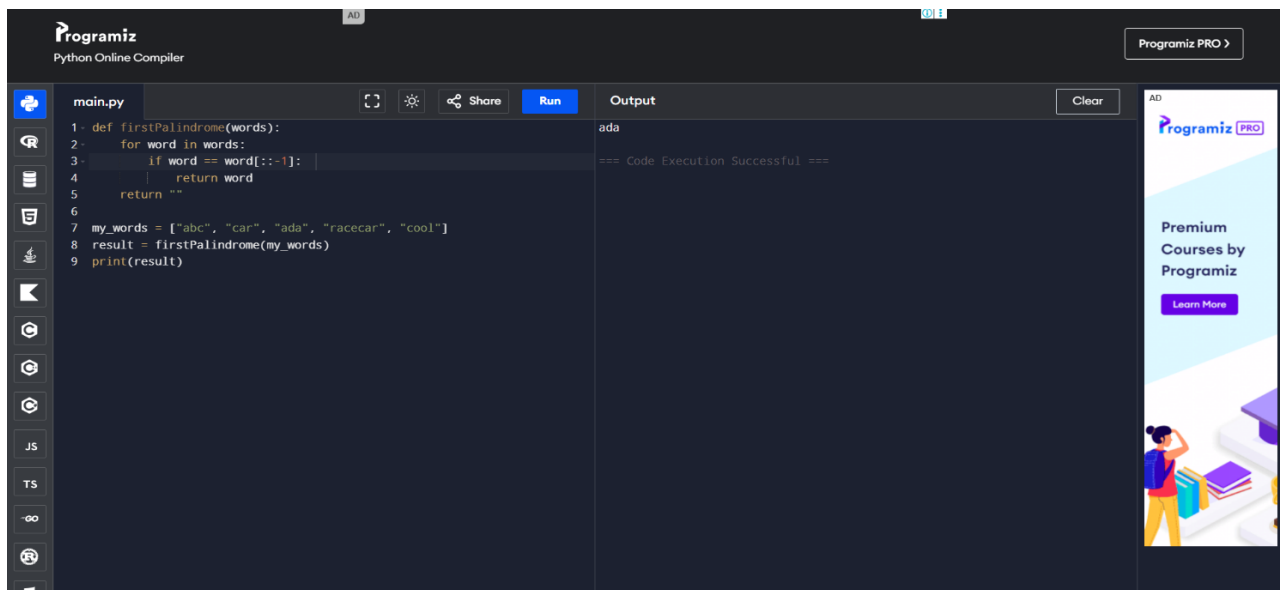
Aim

To write a Python program to find the first palindromic string in an array of strings.

Algorithm

1. Read the list of strings.
2. Traverse each string in the list.
3. Reverse the current string.
4. Compare the original string with the reversed string.
5. If both are equal, print the string and stop.
6. If no palindrome is found, return an empty string.

Output



```
main.py
1- def firstPalindrome(words):
2-     for word in words:
3-         if word == word[::-1]:
4-             return word
5-     return ""
6-
7- my_words = ["abc", "car", "ada", "racecar", "cool"]
8- result = firstPalindrome(my_words)
9- print(result)
```

Output

```
ada
=== Code Execution Successful ===
```

Result

Thus, the first palindromic string in the given array was found successfully.

Time Complexity

$O(n \times m)$

Space Complexity

$O(1)$

Experiment 2 – Count Common Elements Between Two Arrays

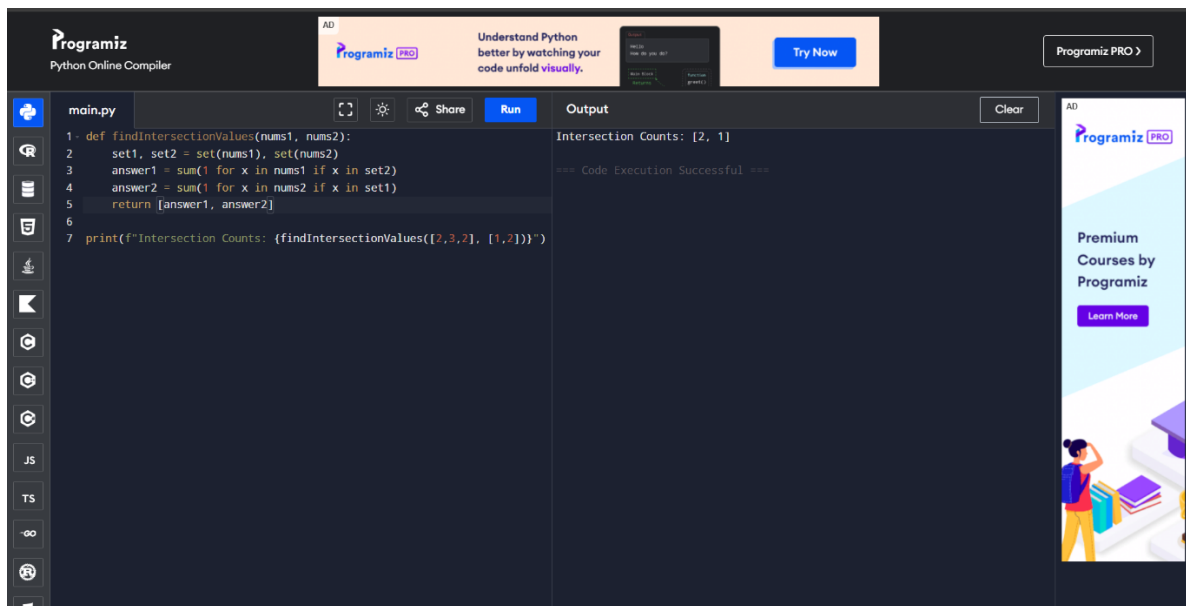
Aim

To write a Python program to count how many elements of one array exist in another array.

Algorithm

1. Read both arrays.
2. Convert both arrays into sets.
3. Traverse the first array and count elements present in the second set.
4. Traverse the second array and count elements present in the first set.
5. Print both counts.

Output



```
1 def findIntersectionValues(nums1, nums2):
2     set1, set2 = set(nums1), set(nums2)
3     answer1 = sum(1 for x in nums1 if x in set2)
4     answer2 = sum(1 for x in nums2 if x in set1)
5     return [answer1, answer2]
6
7 print(f"Intersection Counts: {findIntersectionValues([2,3,2], [1,2])}")
```

Intersection Counts: [2, 1]

=== Code Execution Successful ===

Result

Thus, the number of common elements between both arrays was found successfully.

Time Complexity

$O(n + m)$

Space Complexity

$O(n + m)$

Experiment 3 – Sum of Squares of Distinct Counts of All Subarrays

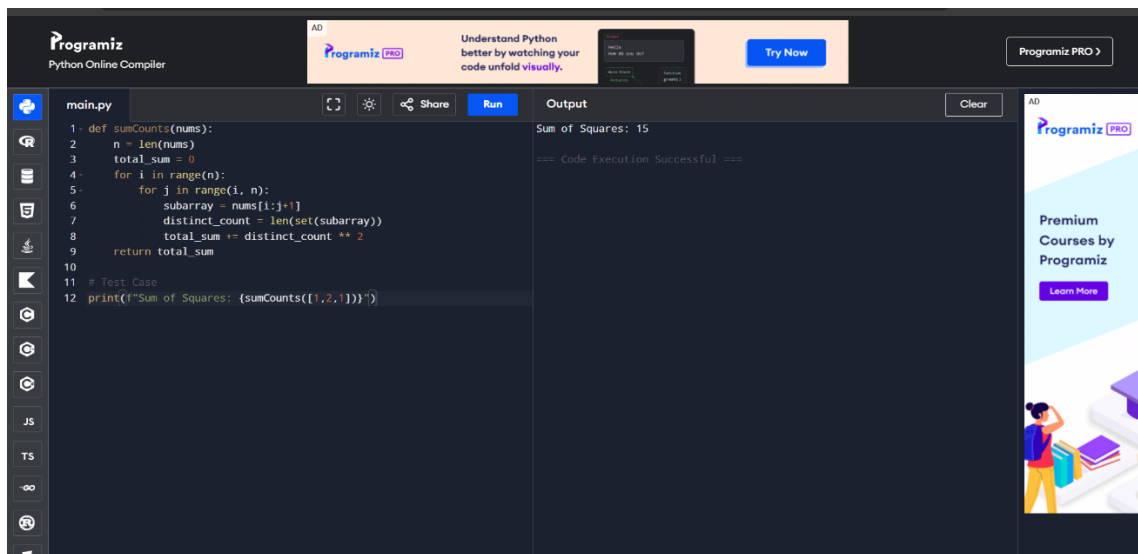
Aim

To write a Python program to find the sum of squares of distinct element counts of all subarrays.

Algorithm

1. Read the input array.
2. Generate every possible subarray.
3. Find the distinct elements using a set.
4. Compute the square of the distinct count.
5. Add the value to the total sum.
6. Display the final sum.

Output



```
1- def sumCounts(nums):
2-     n = len(nums)
3-     total_sum = 0
4-     for i in range(n):
5-         for j in range(i, n):
6-             subarray = nums[i:j+1]
7-             distinct_count = len(set(subarray))
8-             total_sum += distinct_count ** 2
9-     return total_sum
10
11 # Test Case
12 print(f"Sum of Squares: {sumCounts([1,2,1])}")
```

Sum of Squares: 15

=== Code Execution Successful ===

Result

Thus, the sum of squares of distinct counts of all subarrays was calculated successfully.

Time Complexity

$O(n^2)$

Space Complexity

$O(n)$

Experiment 4 – Count Equal Pairs Divisible by k

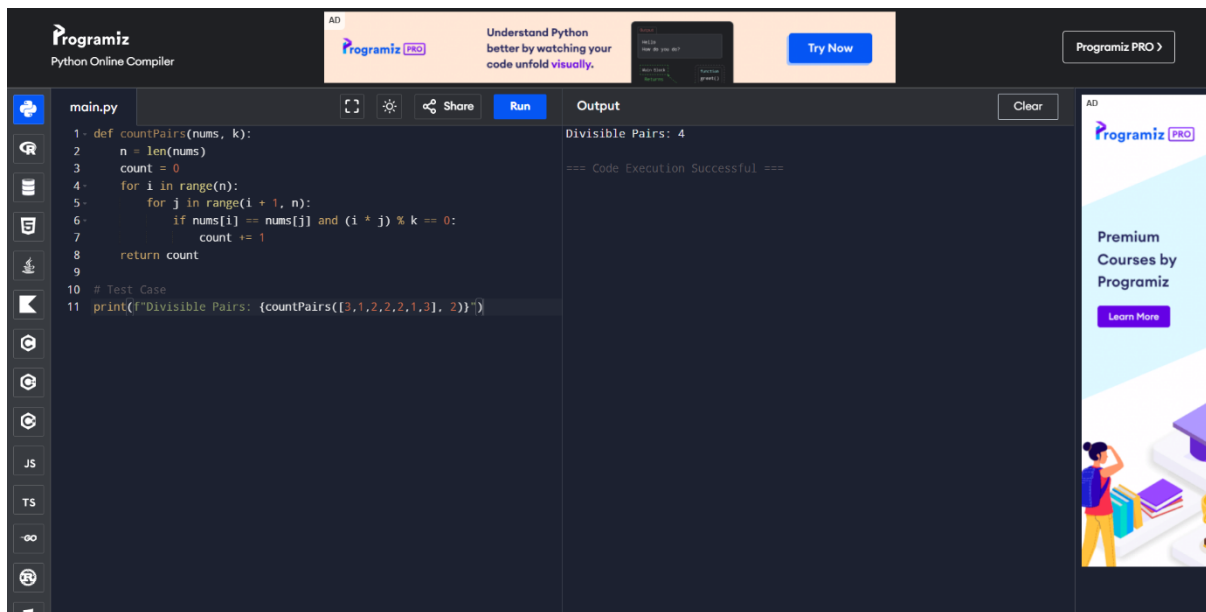
Aim

To write a Python program to count pairs whose values are equal and whose index product is divisible by k.

Algorithm

1. Read the array and value of k.
2. Compare every pair of indices.
3. Check whether both elements are equal.
4. Check whether $(i \times j)$ is divisible by k.
5. Increment the count if both conditions are satisfied.
6. Print the final count.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python function `countPairs` that iterates through an array `nums` and counts pairs `(i, j)` where `nums[i] == nums[j]` and `(i * j) % k == 0`. A test case is provided: `print(f'Divisible Pairs: {countPairs([3,1,2,2,2,1,3], 2)}')`. The output panel on the right displays the result: `Divisible Pairs: 4` and `=== Code Execution Successful ===`. The interface also includes a sidebar with icons for various programming languages and a top navigation bar with a 'Try Now' button.

Result

Thus, the required number of valid pairs was obtained successfully.

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Experiment 5 – Find Maximum Element

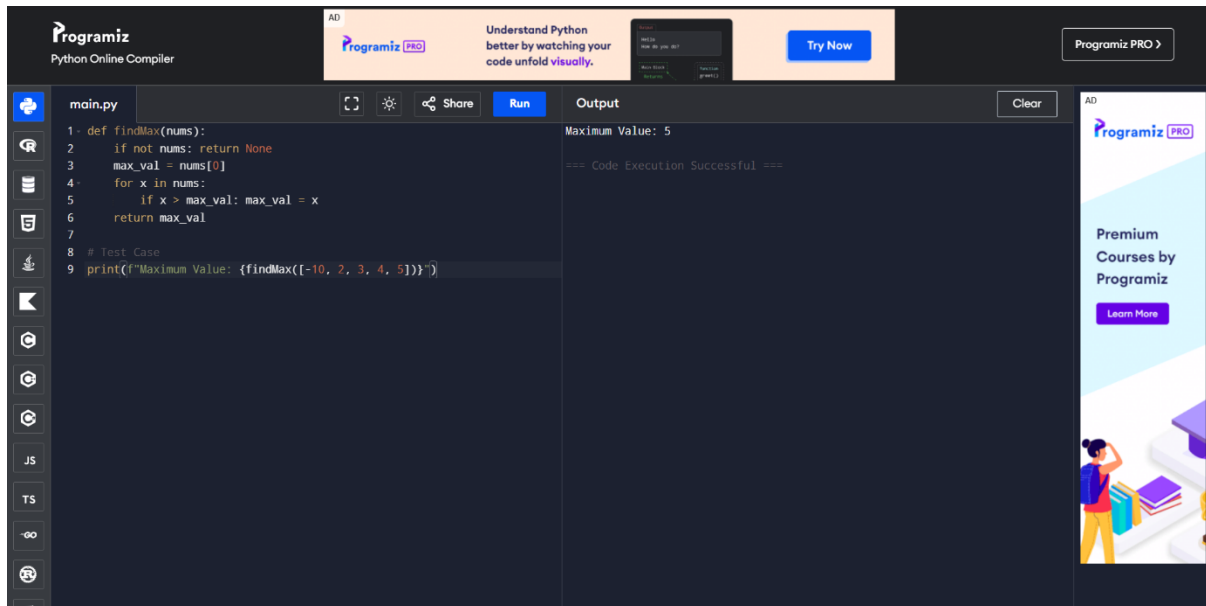
Aim

To write a Python program to find the maximum element in an array.

Algorithm

1. Read the array.
2. Assume the first element as maximum.
3. Compare every remaining element.
4. Update the maximum whenever a larger element is found.
5. Display the maximum element.

Output



```
1 def findMax(nums):
2     if not nums: return None
3     max_val = nums[0]
4     for x in nums:
5         if x > max_val: max_val = x
6     return max_val
7
8 # Test Case
9 print(f"Maximum Value: {findMax([-10, 2, 3, 4, 5])}")
```

Maximum Value: 5

=== Code Execution Successful ===

Result

Thus, the maximum element of the array was found successfully.

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Experiment 6 – Sort the List and Find Maximum Element

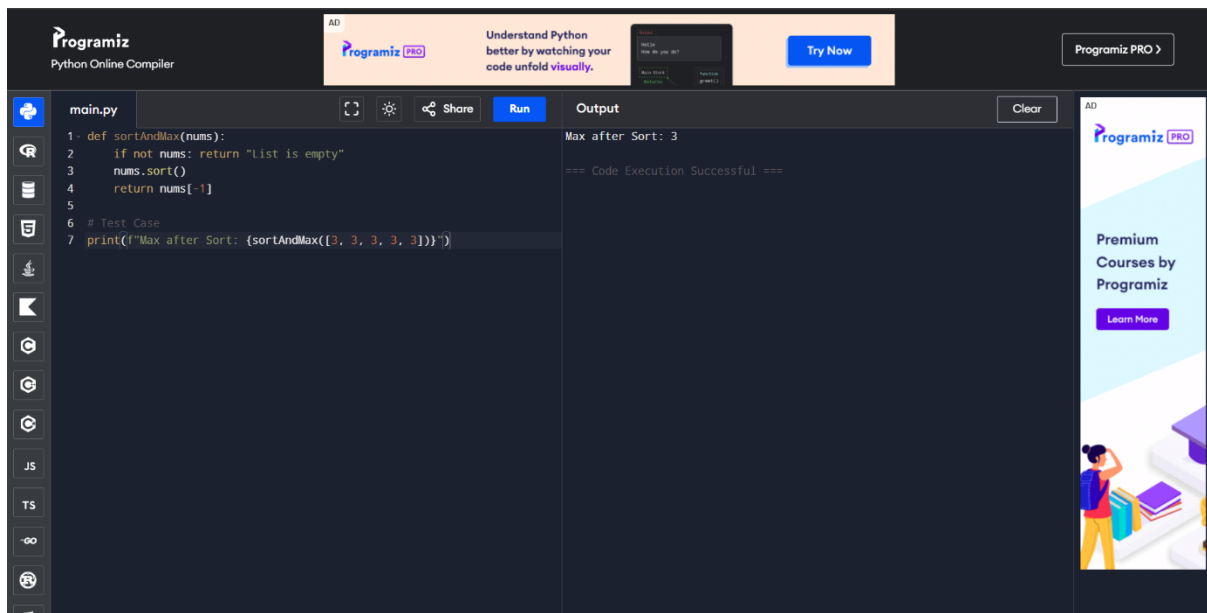
Aim

To write a Python program to sort the list and then find the maximum element.

Algorithm

1. Read the input list.
2. Sort the list using Merge Sort.
3. Check whether the list is empty.
4. If empty, display an appropriate message.
5. Otherwise, print the last element of the sorted list.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1. def sortAndMax(nums):
2.     if not nums: return "List is empty"
3.     nums.sort()
4.     return nums[-1]
5.
6. # Test Case
7. print("Max after Sort: {sortAndMax([3, 3, 3, 3, 3])}")
```

The output panel on the right displays the result of the code execution:

```
Max after Sort: 3
=== Code Execution Successful ===
```

The interface also includes a top navigation bar with the Programiz logo, a "Try Now" button, and a "Programiz PRO" button. A sidebar on the right promotes "Premium Courses by Programiz" with a "Learn More" button.

Result

Thus, the list was sorted successfully and the maximum element was found.

Time Complexity

$O(n \log n)$

Space Complexity

$O(n)$

Experiment 7 – Find Unique Elements

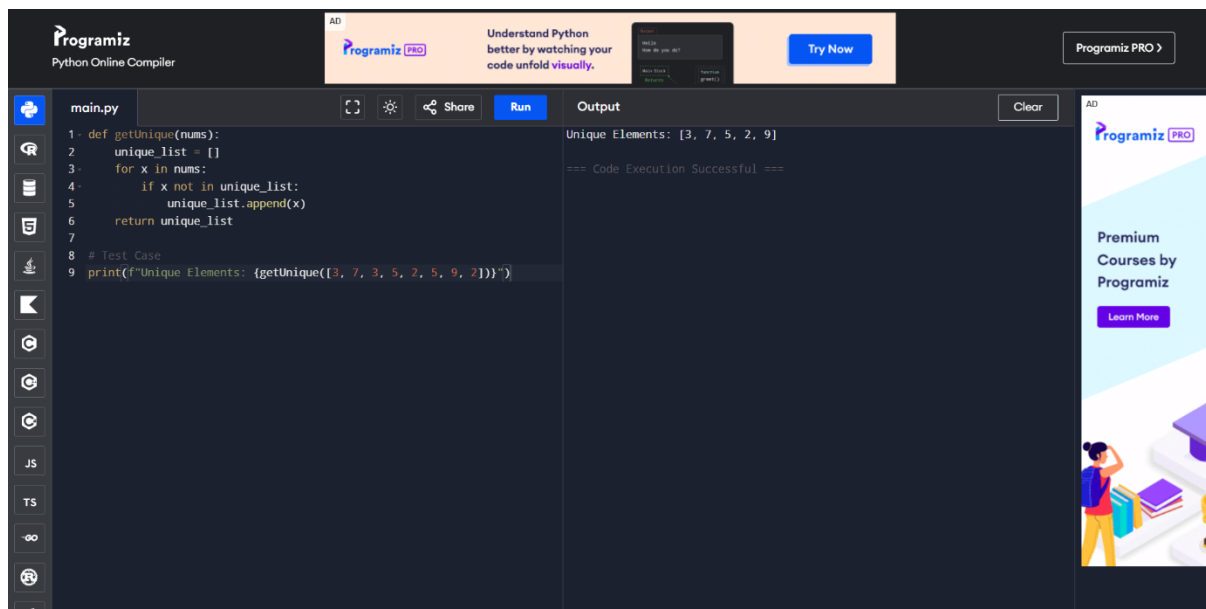
Aim

To write a Python program to create a list containing only unique elements.

Algorithm

1. Read the list.
2. Create an empty set.
3. Traverse each element.
4. If the element is not present in the set, add it to the result list.
5. Display the unique list.

Output



The screenshot displays the Programiz Python Online Compiler interface. On the left, a file explorer shows 'main.py'. The code editor contains the following Python code:

```
1 def getUnique(nums):
2     unique_list = []
3     for x in nums:
4         if x not in unique_list:
5             unique_list.append(x)
6     return unique_list
7
8 # Test Case
9 print(f"Unique Elements: {getUnique([3, 7, 3, 5, 2, 5, 9, 2])}")
```

On the right, the 'Output' panel shows the result of the code execution:

```
Unique Elements: [3, 7, 5, 2, 9]
=== Code Execution Successful ===
```

The interface also includes a top navigation bar with the Programiz logo, a 'Try Now' button, and a 'Programiz PRO' badge. A sidebar on the right promotes 'Premium Courses by Programiz' with a 'Learn More' button.

Result

Thus, the unique elements were extracted successfully.

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Experiment 8 – Bubble Sort

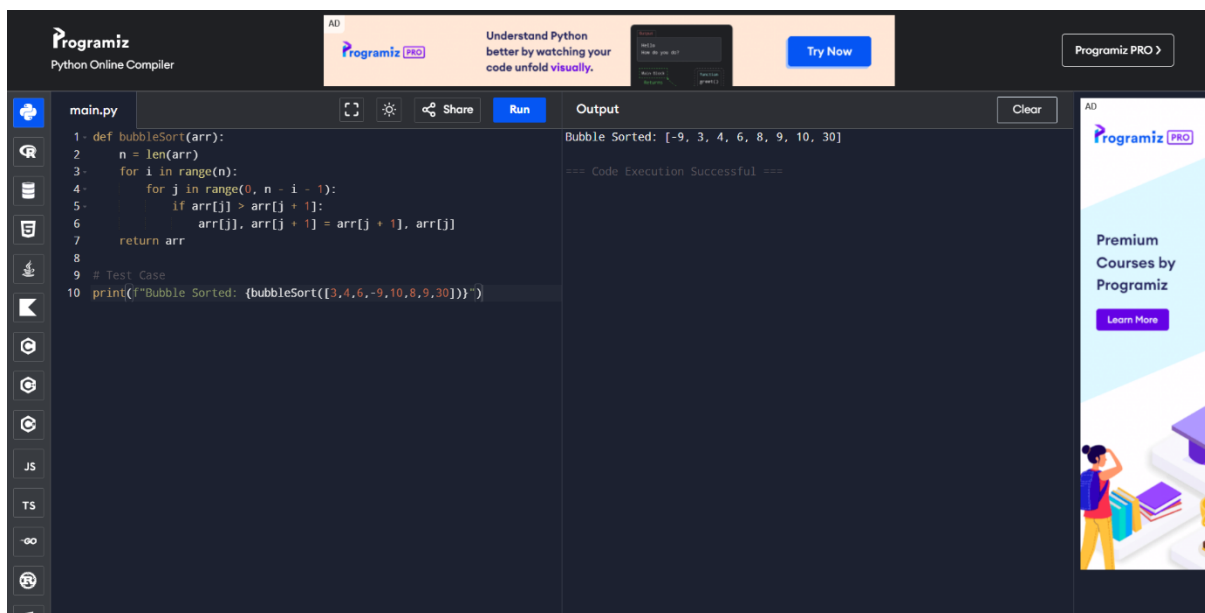
Aim

To write a Python program to sort an array using Bubble Sort.

Algorithm

1. Read the array.
2. Compare adjacent elements.
3. Swap them if they are in the wrong order.
4. Repeat until the array becomes sorted.
5. Display the sorted array.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python program for Bubble Sort. The output window on the right displays the sorted array: [-9, 3, 4, 6, 8, 9, 10, 30]. The code is as follows:

```
1 def bubbleSort(arr):
2     n = len(arr)
3     for i in range(n):
4         for j in range(0, n - i - 1):
5             if arr[j] > arr[j + 1]:
6                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
7     return arr
8
9 # Test Case
10 print(f"Bubble Sorted: {bubbleSort([3,4,6,-9,10,8,9,30])}")
```

The output window shows: Bubble Sorted: [-9, 3, 4, 6, 8, 9, 10, 30] and === Code Execution Successful ===. The interface also includes a sidebar with icons for various programming languages and a top banner for Programiz PRO.

Result

Thus, the array was sorted successfully using Bubble Sort.

Time Complexity

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

$O(1)$

Experiment 9 – Binary Search

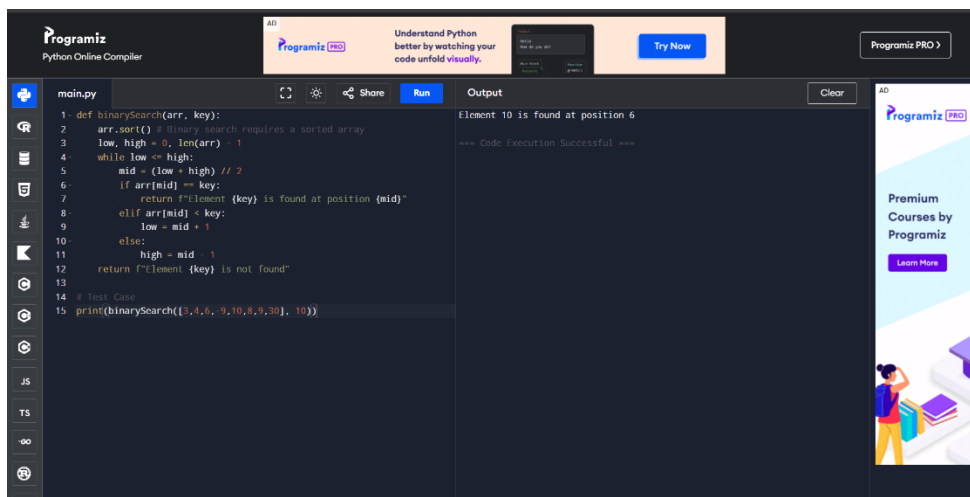
Aim

To write a Python program to search for a given element in a sorted array using the Binary Search algorithm.

Algorithm

1. Read the sorted array and the key element.
2. Initialize `low = 0` and `high = n-1`.
3. Find the middle element.
4. If the middle element equals the key, return its position.
5. If the key is smaller, search the left half.
6. Otherwise, search the right half.
7. Repeat until the element is found or the search interval becomes empty.
8. Display the result.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python function `binarySearch(arr, key)` that implements the binary search algorithm. The function sorts the array, initializes `low` and `high`, and uses a `while` loop to find the element. The output window on the right displays the result: "Element 10 is found at position 6".

```
1 def binarySearch(arr, key):
2     arr.sort() # Binary search requires a sorted array
3     low, high = 0, len(arr) - 1
4     while low <= high:
5         mid = (low + high) // 2
6         if arr[mid] == key:
7             return "Element {key} is found at position {mid}"
8         elif arr[mid] < key:
9             low = mid + 1
10        else:
11            high = mid - 1
12    return f"Element {key} is not found"
13
14 # Test Case
15 print(binarySearch([3,4,6, 9,10,8,9,30], 10))
```

Output: Element 10 is found at position 6

Code Execution Successful

Result

Thus, the given element was searched successfully using Binary Search.

Time Complexity

- Best Case: **$O(1)$**
- Average Case: **$O(\log n)$**
- Worst Case: **$O(\log n)$**

Space Complexity

- Iterative: **$O(1)$**
- Recursive: **$O(\log n)$**

Experiment 10 – Merge Sort

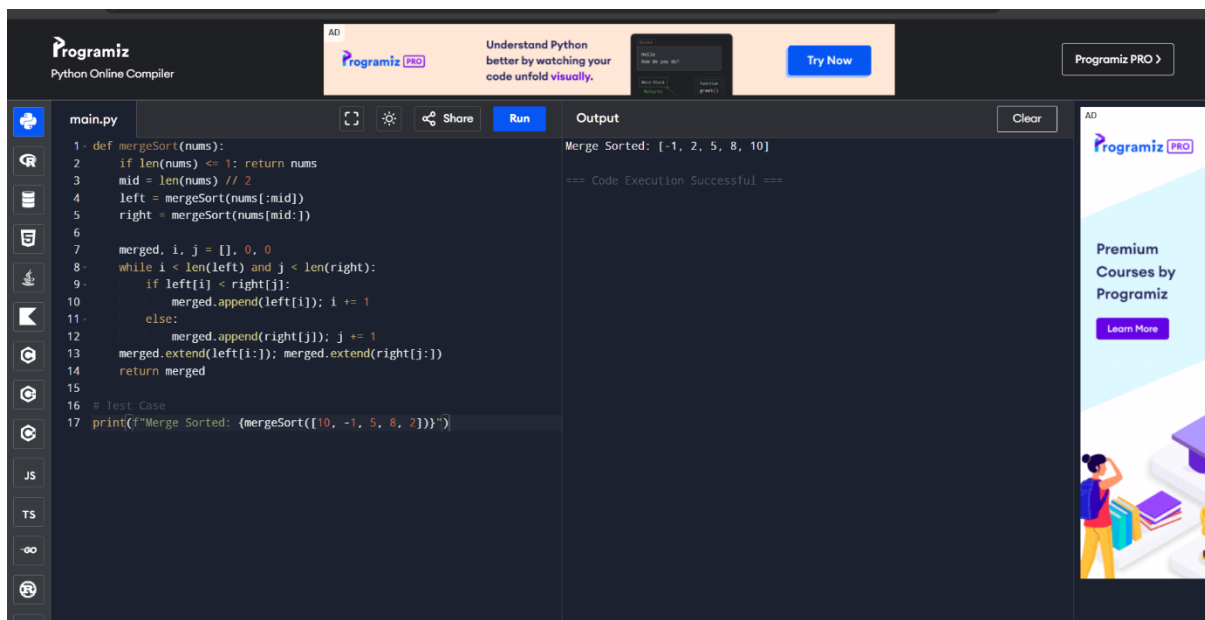
Aim

To write a Python program to sort an array in ascending order using the Merge Sort algorithm.

Algorithm

1. Read the input array.
2. Divide the array into two equal halves.
3. Recursively sort each half.
4. Merge the sorted halves into a single sorted array.
5. Display the sorted array.

Output



The screenshot shows the Programiz Python Online Compiler interface. On the left, a file explorer shows 'main.py'. The code editor contains a Python function 'mergeSort(nums)' that recursively sorts an array. The function uses a divide-and-conquer approach: it checks if the array length is 1 or less, then splits the array into two halves, recursively sorts them, and finally merges them back into a single sorted array. A test case is provided at the bottom: `print(f'Merge Sorted: {mergeSort([10, -1, 5, 8, 2])}')`. On the right, the 'Output' panel shows the result: 'Merge Sorted: [-1, 2, 5, 8, 10]' and a message '=== Code Execution Successful ==='. The interface also includes a 'Run' button, a 'Clear' button, and a sidebar with various icons and a 'Programiz PRO' banner.

```
1 def mergeSort(nums):
2     if len(nums) <= 1: return nums
3     mid = len(nums) // 2
4     left = mergeSort(nums[:mid])
5     right = mergeSort(nums[mid:])
6
7     merged, i, j = [], 0, 0
8     while i < len(left) and j < len(right):
9         if left[i] < right[j]:
10            merged.append(left[i]); i += 1
11        else:
12            merged.append(right[j]); j += 1
13    merged.extend(left[i:]); merged.extend(right[j:])
14    return merged
15
16 # Test Case
17 print(f'Merge Sorted: {mergeSort([10, -1, 5, 8, 2])}')
```

Merge Sorted: [-1, 2, 5, 8, 10]

=== Code Execution Successful ===

Result

Thus, the array was sorted successfully using Merge Sort.

Time Complexity

- Best Case: **$O(n \log n)$**
- Average Case: **$O(n \log n)$**
- Worst Case: **$O(n \log n)$**

Space Complexity

$O(n)$

Experiment 11 – Out of Boundary Paths

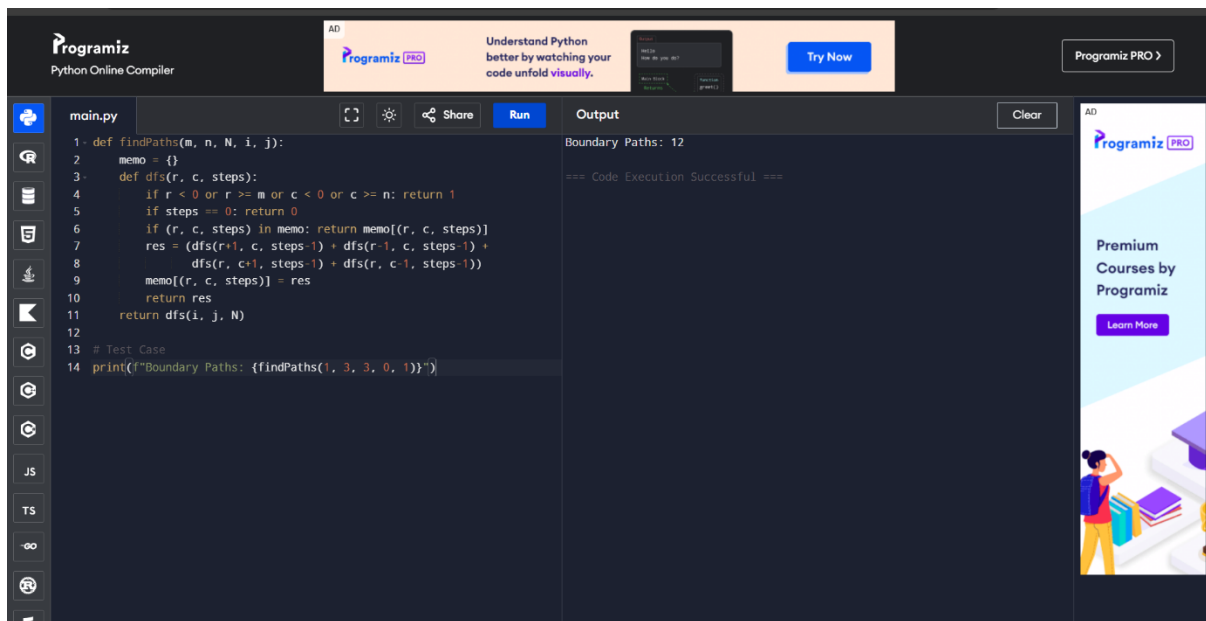
Aim

To write a Python program to find the number of ways a ball can move out of a grid boundary in exactly N steps.

Algorithm

1. Read the values of m, n, N, starting row, and starting column.
2. Use Dynamic Programming to store intermediate results.
3. For each move, calculate all four possible directions.
4. Count the paths that move outside the grid.
5. Repeat until N moves are completed.
6. Display the total number of paths.

Output



```
1 def findPaths(m, n, N, i, j):
2     memo = {}
3     def dfs(r, c, steps):
4         if r < 0 or r >= m or c < 0 or c >= n: return 1
5         if steps == 0: return 0
6         if (r, c, steps) in memo: return memo[(r, c, steps)]
7         res = (dfs(r+1, c, steps-1) + dfs(r-1, c, steps-1) +
8               dfs(r, c+1, steps-1) + dfs(r, c-1, steps-1))
9         memo[(r, c, steps)] = res
10        return res
11    return dfs(i, j, N)
12
13 # Test Case
14 print(f"Boundary Paths: {findPaths(1, 3, 3, 0, 1)}")
```

Boundary Paths: 12

=== Code Execution Successful ===

Result

Thus, the number of paths to move the ball outside the grid was found successfully.

Time Complexity

$O(m \times n \times N)$

Space Complexity

$O(m \times n)$

Experiment 12 – House Robber II

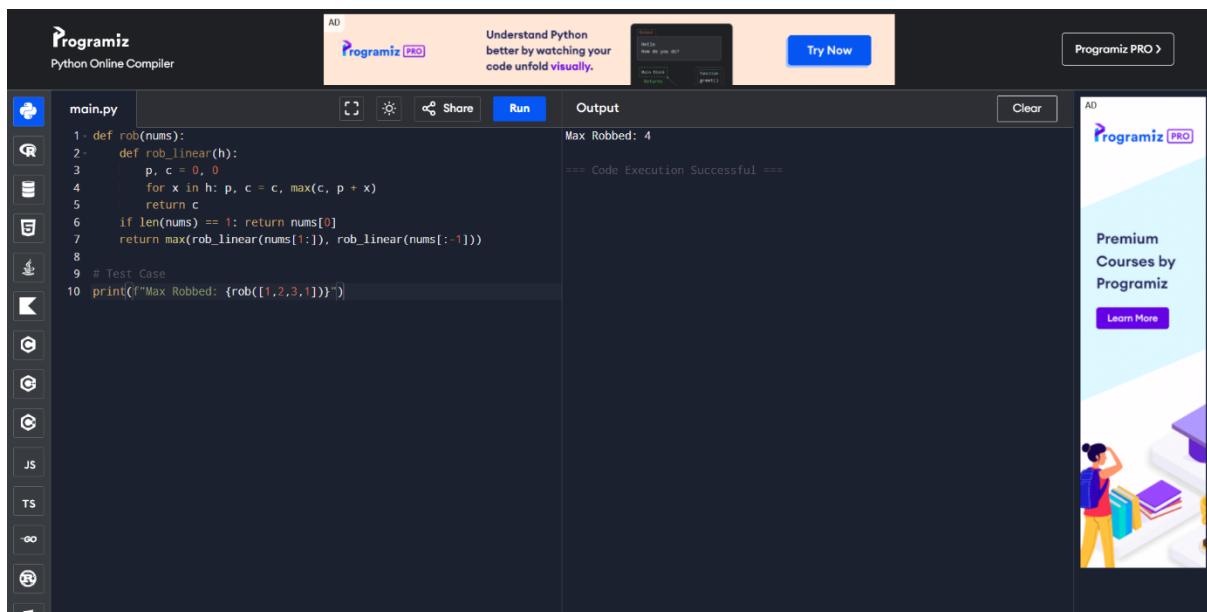
Aim

To write a Python program to find the maximum money that can be robbed from circularly arranged houses without robbing adjacent houses.

Algorithm

1. Read the money available in each house.
2. Since the houses are circular, consider two cases:
 - Exclude the first house.
 - Exclude the last house.
3. Apply Dynamic Programming to both cases.
4. Find the maximum amount for each case.
5. Print the larger value.

Output



The screenshot displays the Programiz Python Online Compiler interface. The code editor on the left contains a Python script named `main.py` that implements a dynamic programming solution for the House Robber II problem. The script defines a `rob_linear` function to handle a linear array of houses and a `rob` function to handle the circular arrangement by comparing the results of excluding the first and last houses. A test case is provided: `print(f"Max Robbed: {rob([1,2,3,1])}")`. The output pane on the right shows the result: `Max Robbed: 4` and `=== Code Execution Successful ===`. The interface also includes a sidebar with icons for file operations and a top navigation bar with a 'Try Now' button and a 'Programiz PRO' badge.

```
1 def rob(nums):
2     def rob_linear(h):
3         p, c = 0, 0
4         for x in h: p, c = c, max(c, p + x)
5         return c
6     if len(nums) == 1: return nums[0]
7     return max(rob_linear(nums[1:]), rob_linear(nums[:-1]))
8
9 # Test Case
10 print(f"Max Robbed: {rob([1,2,3,1])}")
```

Max Robbed: 4
=== Code Execution Successful ===

Result

Thus, the maximum amount of money that can be robbed was calculated successfully.

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Experiment 13 – Climbing Stairs

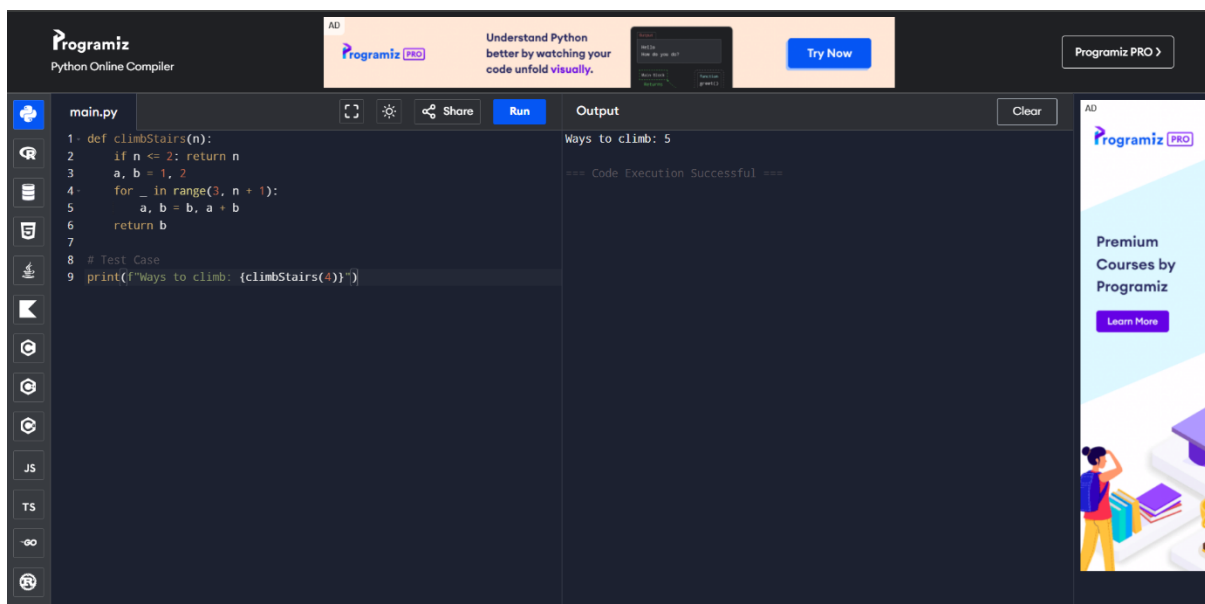
Aim

To write a Python program to find the number of distinct ways to climb a staircase.

Algorithm

1. Read the number of steps.
2. Initialize the first two values.
3. Compute each new value as the sum of the previous two values.
4. Continue until the required step is reached.
5. Display the total number of ways.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1. def climbStairs(n):
2.     if n <= 2: return n
3.     a, b = 1, 2
4.     for _ in range(3, n + 1):
5.         a, b = b, a + b
6.     return b
7.
8. # Test Case
9. print(f'Ways to climb: {climbStairs(4)}')
```

The output panel on the right displays the result of the code execution:

```
Ways to climb: 5
==> Code Execution Successful ==>
```

The interface also includes a sidebar with icons for various programming languages (Python, JS, TS, etc.) and a top navigation bar with the Programiz logo and a 'Try Now' button.

Result

Thus, the number of distinct ways to climb the staircase was calculated successfully.

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Experiment 14 – Unique Paths

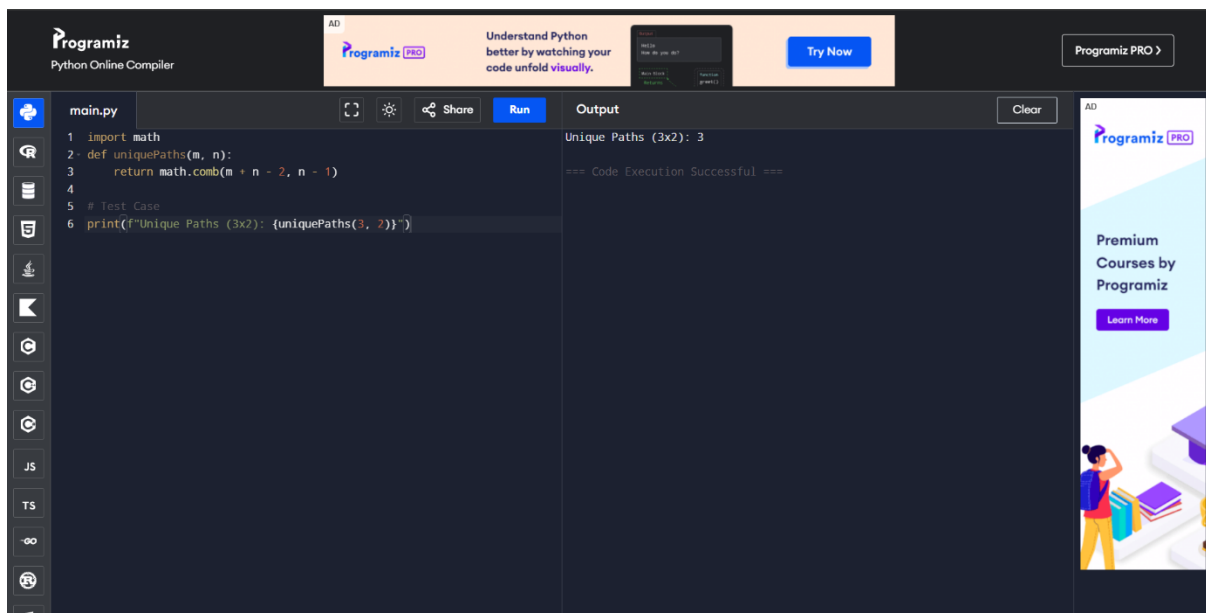
Aim

To write a Python program to find the number of unique paths from the top-left corner to the bottom-right corner of a grid.

Algorithm

1. Read the values of m and n.
2. Create a Dynamic Programming table.
3. Fill the first row and first column with 1.
4. Compute each remaining cell as the sum of its top and left cells.
5. Display the value in the last cell.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1 import math
2 def uniquePaths(m, n):
3     return math.comb(m + n - 2, n - 1)
4
5 # Test Case
6 print(f"Unique Paths (3x2): {uniquePaths(3, 2)}")
```

The output panel on the right displays the result of the code execution:

```
Unique Paths (3x2): 3
=== Code Execution Successful ===
```

The interface also includes a top navigation bar with the Programiz logo, a search bar, and a 'Try Now' button. A sidebar on the right promotes 'Premium Courses by Programiz' with a 'Learn More' button.

Result

Thus, the number of unique paths was calculated successfully.

Time Complexity

$O(m \times n)$

Space Complexity

$O(m \times n)$

Experiment 15 – Large Group Positions

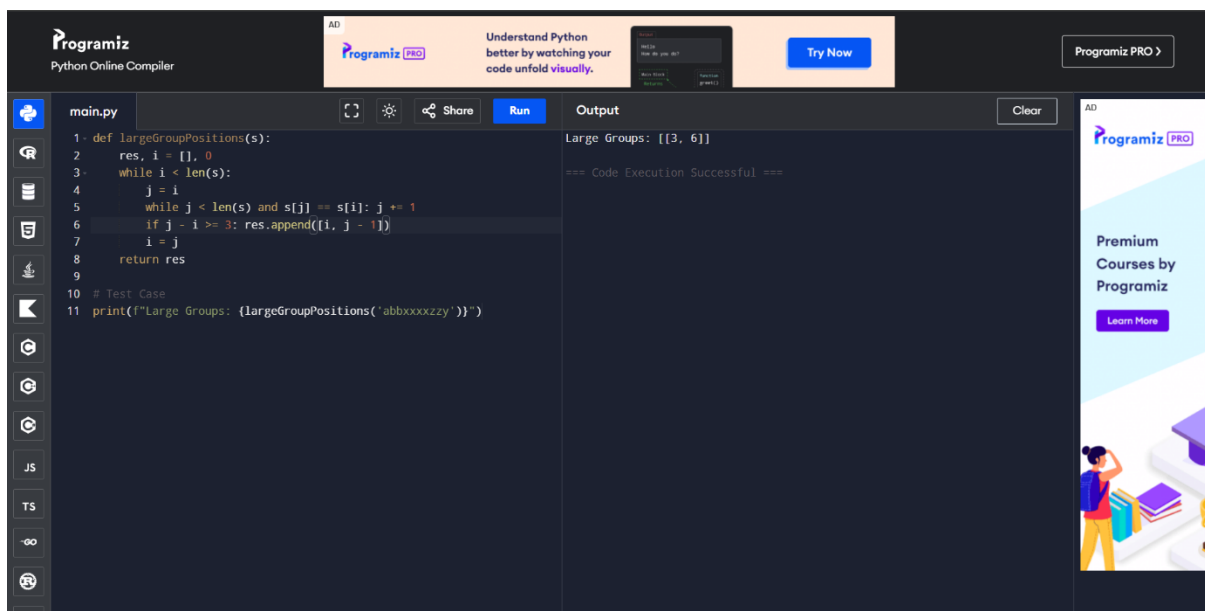
Aim

To write a Python program to identify all large groups of consecutive characters in a string.

Algorithm

1. Read the input string.
2. Traverse the string from left to right.
3. Count consecutive occurrences of each character.
4. If the count is greater than or equal to 3, store the starting and ending indices.
5. Display all such intervals.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1 def largeGroupPositions(s):
2     res, i = [], 0
3     while i < len(s):
4         j = i
5         while j < len(s) and s[j] == s[i]: j += 1
6         if j - i >= 3: res.append([i, j - 1])
7         i = j
8     return res
9
10 # Test Case
11 print(f"Large Groups: {largeGroupPositions('abbxxxxzzy')}")
```

The output panel on the right displays the result of the code execution:

```
Large Groups: [[3, 6]]
=== Code Execution Successful ===
```

The interface also includes a sidebar with icons for various programming languages (Python, JS, TS, etc.) and a top navigation bar with the Programiz logo and a 'Try Now' button.

Result

Thus, all the large groups in the given string were identified successfully.

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Experiment 16 – Game of Life

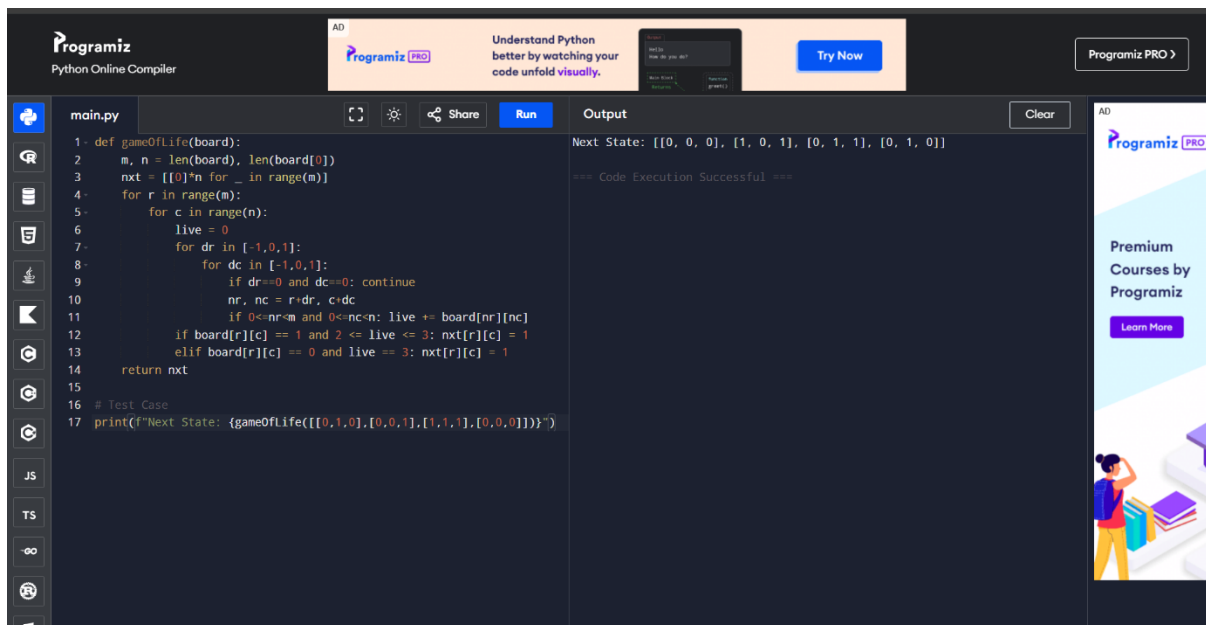
Aim

To write a Python program to generate the next state of the Game of Life based on Conway's rules.

Algorithm

1. Read the input grid.
2. For each cell, count its live neighbors.
3. Apply Conway's Game of Life rules.
4. Store the updated state in a temporary grid.
5. Replace the original grid with the updated grid.
6. Display the next generation.

Output



The screenshot displays the Programiz Python Online Compiler interface. On the left, a file explorer shows 'main.py'. The main editor contains a Python function `gameOfLife(board)` that implements Conway's Game of Life rules. The function iterates through each cell of the input grid, counts its live neighbors, and updates the next state based on the rules. A test case is provided at the bottom: `print(f'Next State: {gameOfLife([[0,1,0],[0,0,1],[1,1,1],[0,0,0]])}')'`. On the right, the 'Output' panel shows the result: `Next State: [[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]`, followed by the message '=== Code Execution Successful ==='. The interface also includes a 'Run' button, a 'Share' icon, and a 'Clear' button for the output.

Result

Thus, the next generation of the Game of Life was generated successfully.

Time Complexity

$O(m \times n)$

Space Complexity

$O(m \times n)$

Experiment 17 – Champagne Tower

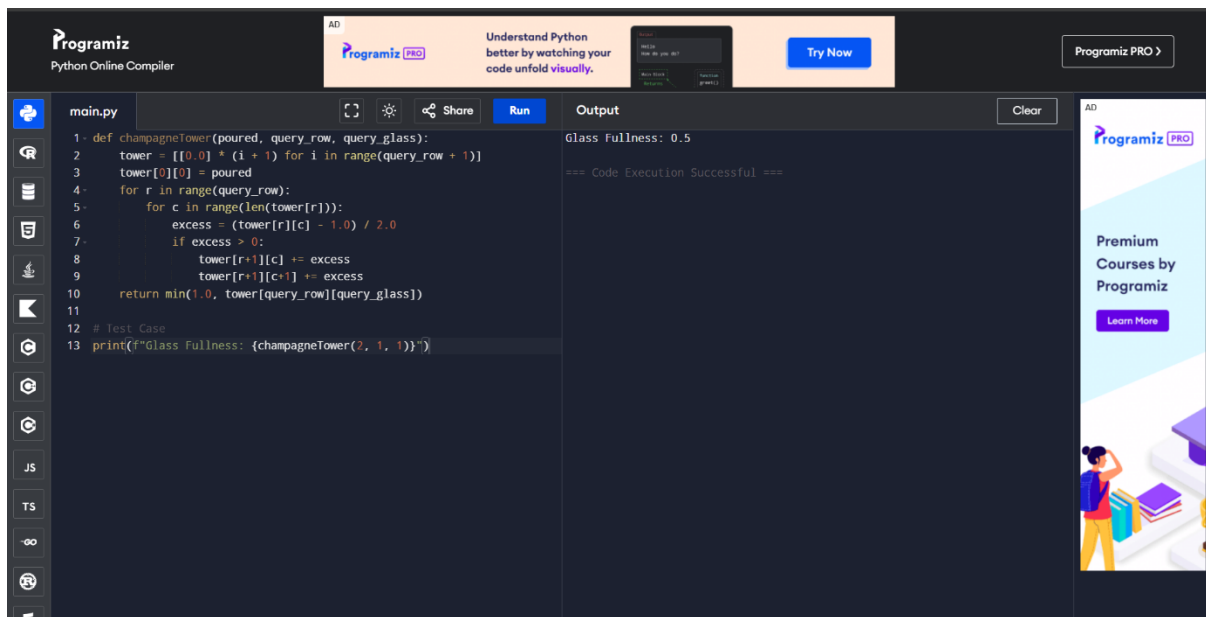
Aim

To write a Python program to determine how full a specific glass is after pouring champagne into a pyramid of glasses.

Algorithm

1. Read the poured amount, query row, and query glass.
2. Create a Dynamic Programming table for the champagne tower.
3. Pour the champagne into the top glass.
4. Distribute excess champagne equally to the two glasses below.
5. Continue until the required row is reached.
6. Display the amount of champagne in the required glass.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python program named `main.py` that defines a function `champagneTower` to calculate the champagne level in a specific glass of a pyramid. The program includes a test case that calls the function with `2, 1, 1`. The output panel on the right shows the result: `Glass Fullness: 0.5` and `=== Code Execution Successful ===`. The interface also features a sidebar with icons for various programming languages and a top navigation bar with a 'Try Now' button.

```
1- def champagneTower(poured, query_row, query_glass):
2-     tower = [[0.0] * (i + 1) for i in range(query_row + 1)]
3-     tower[0][0] = poured
4-     for r in range(query_row):
5-         for c in range(len(tower[r])):
6-             excess = (tower[r][c] - 1.0) / 2.0
7-             if excess > 0:
8-                 tower[r+1][c] += excess
9-                 tower[r+1][c+1] += excess
10-    return min(1.0, tower[query_row][query_glass])
11
12 # Test Case
13 print(f"Glass Fullness: {champagneTower(2, 1, 1)}")
```

Glass Fullness: 0.5
=== Code Execution Successful ===

Result

Thus, the amount of champagne in the specified glass was calculated successfully.

Time Complexity

$O(r^2)$ (where r is the query row)

Space Complexity

$O(r^2)$